Codex CLI 讲义版摘要

适合培训、复习、分享的速读版

第一次上手

权限与审批

提示词速记

这份短版保留了最核心的起步路径：安装、提问、权限判断、常用命令、稳定工作流、排错顺序和最后的速记卡。

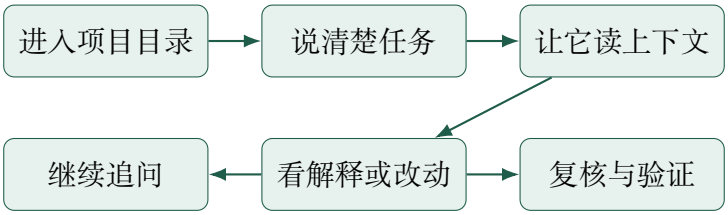
请以你本地 `codex --help` 和 OpenAI 官方 Codex 文档为准。

1 一页理解

先记住一句最实用的话：**Codex CLI 是一个在终端里工作的 AI 编码代理。**

它最适合做四类事：

- 看懂陌生项目；
- 定位功能和报错；
- 做小范围改动；
- 做 review、总结、验证建议。



速记

第一次用 Codex CLI，不求全会。先掌握一条线：进目录、提具体任务、继续追问、最后验证。

2 第一次上手

2.1 安装与登录

```
npm i -g @openai/codex
codex --help
codex
```

如果你想显式登录或检查状态：

```
codex login
codex login status
```

如果你走 API key 路线：

```
printenv OPENAI_API_KEY | codex login --with-api-key
```

2.2 第一次提问

> 请先看一下当前项目结构，告诉我入口文件、核心目录和推荐阅读顺序，并请引用具体文件路径。

别踩这个坑

不要一上来就说“帮我改好”或“帮我优化一下”。范围太宽，结果通常会变空。

3 最常用命令

命令	作用	什么时候用
`codex`	交互式会话	本地日常开发最常用
`codex exec "..."`	非交互执行	脚本、CI（Continuous Integration，持续集成）、一次性任务
`codex review --uncommitted`	审查当前改动	提交前快速找风险
`codex resume --last`	接着上次会话干	不想重复讲背景时
`codex apply <TASK_ID>`	应用某次任务 diff	结果要落到工作树时
`/status`	查看会话状态	看模型、权限、可写目录
`/permissions`	调整权限模式	切换更保守或更自动
`/review`	会话内做 review	先查风险再看 diff
`/diff`	查看改动	改完先审一眼
`/plan`	先出计划	大任务先计划再动手
`/init`	生成 `AGENTS.md` 脚手架	想沉淀仓库规则时

记住这一点

如果你只背三个入口，先背 `codex`、`codex exec`、`codex review`。

4 权限与审批

4.1 三个沙箱级别

- `read-only`：只读，不改。
- `workspace-write`：能改当前工作区，最适合本地开发。
- `danger-full-access`：权限很大，只在你非常清楚风险时使用。

4.2 三个审批策略

- `untrusted`：安全读操作自动跑，不信任命令先问你。
- `on-request`：由代理判断什么时候该申请批准。
- `never`：不弹审批，常用于自动化，但前提是你自己已经控好风险。

```
codex --full-auto
```

它相当于一个比较实用的低摩擦默认组合：

```
--sandbox workspace-write --ask-for-approval on-request
```

速记

本地开发优先从 `workspace-write` 起步。先别急着全放开。

5 提示词速记

5.1 一条好提示词的四要素

1. 目标：你到底要它做什么。
2. 范围：只围绕哪些文件、功能或问题。
3. 限制：不要做什么。
4. 输出格式：你希望它最后怎么汇报。

5.2 直接可用模板

看项目：

```
请按新手视角介绍当前项目，重点说入口、核心目录、主流程和推荐阅读顺序。
```

定位功能：

```
请帮我定位登录相关代码，告诉我入口、涉及文件和大致调用链。
```

修 bug：

```
这是报错信息。请先解释最可能原因，再给排查顺序，最后给最小修改方案。
```

小改动：

```
请只修改提示文案，不要改业务逻辑。
修改后列出改动文件和验证方法。
```

6 稳定工作流

6.1 本地开发推荐顺序

1. 先解释项目或问题；
2. 再定位文件和调用链；
3. 再做小范围修改；
4. 改完跑 `/review`；
5. 再看 `/diff`；
6. 最后自己验证。

6.2 为什么要用 AGENTS.md

`AGENTS.md` 适合沉淀这些长期规则：`lint` 一般就是静态检查，用来提前发现格式、风格和一些明显代码问题。

- 仓库结构和重点目录；
- 构建、测试、lint 命令；
- 哪些文件不要动；
- 交付时怎么总结。

```
/init
```

记住这一点

如果你在三次任务里都重复说过同一句话，那它大概率就该进 `AGENTS.md` 了。

7 自动化与 CI

7.1 最小示例

```
export CODEX_API_KEY=your_api_key

codex exec --json \
  -o codex-summary.txt \
  "Review the current repository and summarize the top 3 risks"
```

7.2 这三个参数分别干什么

- `CODEX_API_KEY`：给非交互执行做认证。
- `--json`：让输出更适合脚本消费。
- `-o`：把最后一条摘要单独落盘。

别踩这个坑

自动化更适合做总结、分类、只读检查。大范围自动改代码，先在本地跑稳再说。

8 排错与避坑

8.1 真实排错顺序

当你遇到报错时，最稳的顺序通常是：

1. 先让它解释错误类型；
2. 再让它定位最可能的文件；
3. 再让它给最小修复方案；
4. 最后让它说明验证方法。

8.2 最容易踩的坑

- 没确认当前目录就开始操作。
- 任务太宽，结果很空。
- 一上来就给太大权限。
- 改完不看 `/diff`，也不做验证。
- 在 CI 里直接照搬交互式习惯。

速记

新手最大的问题不是不会敲命令，而是没有形成节奏：确认目录、收紧范围、审改动、做验证。

9 练习题

9.1 练习一

让 Codex 给出一个公开小仓库的入口文件、核心目录和推荐阅读顺序。

9.2 练习二

选一个功能，例如登录或上传，让它解释入口、涉及文件和调用链。

9.3 练习三

只做一个低风险改动，例如改文案、补说明或补一条简单测试。

9.4 练习四

跑一遍：

```
/review
/diff
```

然后判断它给出的 review 是否真的落到了当前改动上。

10 最后只记这几句

3 分钟背完

1. 日常开发多用 `codex`，脚本化多用 `codex exec`。
2. 第一次上手先做最小任务，不要急着大改。
3. 好提示词要写清楚目标、范围、限制、输出格式。
4. 本地开发先从 `workspace-write` 起步。
5. 改完一定先 `/review`，再 `/diff`，最后自己验证。
6. 有重复规则就写进 `AGENTS.md`。